An Efficient and Reconfigurable Post-Quantum Crypto-Processor for SPHINCS+

Tianze Huang, Jiahao Lu, Dongsheng Liu, *Senior Member, IEEE*, Zhixiang Luo, Chi Cheng, *Member, IEEE*, Aobo Li, *Graduate Student Member, IEEE*, Lei Chen, Shuo Yang, Jiaming Zhang, and Xiang Li

***Abstract*— SPHINCS+ is the sole hash-based digital signature scheme among the selected post-quantum cryptography (PQC) in 2022. This algorithm possesses the ability to resist attacks from both classical and quantum computers. Due to the extensive computations and different data widths for various parameters, its hardware implementation faces the weakness of long operation time, large area requirement, and low flexibility. This paper presents an efficient and reconfigurable SPHINCS+ processor. The proposed on-the-fly *WOTS+* public key generation scheme with unified chain address generator accelerated the most time-consuming operations. This optimization achieves efficient resource utilization. A security switch mechanism resolves the bit misalignment among different data widths with resource reduction. Finally, we introduce a grouped subtree and segmented signature streaming scheme. They reduce the memory to 16k bytes. The processor consumes 29410 LUTs, 14090 FFs, 4 BRAMs on Artix-7 FPGA and achieves 1.04×/2.41× ATPs (area-time-product) optimizations in *Sign*/*Verify* with the advantage of supporting all security levels of SPHINCS+.**

***Index Terms*— Post-quantum cryptography, SPHINCS+, hash-based signature, hardware efficiency, reconfigurable processor.**

I. Introduction

QUANTUM computers with Shor's algorithm [1] pose a great threat on traditional public key cryptography [2], [3] such as RSA [4] and elliptic curve digital signature algorithm (ECDSA) [5]. Security systems built on these conventional cryptographic schemes will be cracked easily in the coming quantum era. It's urgent to carry out research on new cryptographic primitives that resist attacks from quantum computers, namely post-quantum cryptography.

The National Institute of Standards and Technology (NIST) launched PQC project in 2016 to solicit quantum-resistant cryptographic algorithms from all over the world. They strive to set a new standard before quantum computers become popular. Starting from February 2016 to date, screenings were carried out approximately every two years in terms of security, resource overhead and performance. Among the original 69 algorithms, Kyber, Dilithium, Falcon and SPHINCS+ [6] were selected in the final competition. Compared with the other three latticed-based algorithms, SPHINCS+ is the sole hash-based signature scheme mainly due to its straightforward operation and promised security. On August 13, 2024, NIST officially established SLH-DSA as a new digital signature standard in FIPS-205. It is the latest version of SPHINCS+ with nearly the same process and parameters. The main difference is that SLH-DSA generates *WOTS+*/*FORS* secret key with different input data.

Hash-based signature schemes (HSS) utilize a hash function as basic operation. Their security completely relies on the selected algorithm [7], [8]. As the winner of Secure Hash Algorithm-3 (SHA-3) standardization competition, KECCAK is believed to own the resistance against various attacks, including collision, pre-image and second pre-image attacks [9]. Therefore, strict security proof can be derived as long as choosing this algorithm in a hash-based signature scheme. This is a crucial reason for why SPHINCS+ was selected.

SPHINCS+ (or SLH-DSA) originates from SPHINCS [10] and Extended Merkle Signature Scheme (XMSS), they are all state-of-the-art hash-based digital signature schemes. Compared with the latter two, a treeless Winternitz one time signature+ (*WOTS+*) [11] public key compression is introduced, which saves a significant amount of computation. Moreover, its stateless signature structure eliminates the requirement to store long-term keys, reducing the risk of key exposure over an extended period of time.

Some software (SW) implementations on SPHINCS+ are proposed concentrating on different purposes. A pipelined signature streaming mechanism is put forward to address the issue of large signature [12], [13]. This solution reduces approximately 60% of storage consumption at the cost of increasing computation time, especially for *Verify*. Geforce

Received 4 March 2024; revised 29 October 2024 and 21 January 2025; accepted 17 February 2025. Date of publication 26 March 2025; date of current version 30 April 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62134002, Grant 62104076, and Grant 62374064; in part by the National Key Research and Development Program of China under Grant 2021YFA0715502 and Grant 2023YFB4502100; in part by the National Key Analog Integrated Circuit Laboratory Project of China under Grant JCKY2021210C004; in part by the Innovation Project of JinYinHu Laboratory under Grant 2024JYH011401; in part by the Interdisciplinary Research Promotion of Huazhong University of Science and Technology (HUST) under Grant 2023JCYJ040; in part by the Postdoctoral Fellowship Program of China Postdoctoral Science Foundation (CPSF) under Grant GZC20240541; in part by the Open Fund of the State Key Laboratory of Spintronics Devices and Technologies under Grant SPL-2406; and in part by the Fund of National Key Laboratory of Multispectral Information Intelligent Processing Technology under Grant 202410487401. This article was recommended by Associate Editor M. O'Neill. *(Corresponding author: Jiahao Lu.)*

Tianze Huang, Jiahao Lu, Dongsheng Liu, Aobo Li, Lei Chen, Shuo Yang, Jiaming Zhang, and Xiang Li are with the School of Integrated Circuits, Huazhong University of Science and Technology, Wuhan 430074, China (e-mail: d202281027@hust.edu.cn; lujiahaohust@foxmail.com; dsliu@mail.hust.edu.cn; liaobo@hust.edu.cn; m202371512@hust.edu.cn; wxkhturf@qq.com; giraffe@hust.edu.cn; lixiang_51@qq.com).

Zhixiang Luo is with the School of Optical and Electronic Information, Huazhong University of Science and Technology, Wuhan 430074, China (e-mail: zhxluo@hust.edu.cn).

Chi Cheng is with the School of Computer Science, China University of Geosciences, Wuhan 430074, China (e-mail: chengchi@cug.edu.cn).

Digital Object Identifier 10.1109/TCSI.2025.3544341

1549-8328 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on September 25,2026 at 09:10:46 UTC from IEEE Xplore. Restrictions apply.

GTX1080 GPU is utilized to parallelize the *Keygen/Sign/Verify* operations [14] and achieves the highest speeds of 61134/2065/57894 operations per second (ops) to date. However, their single-operation latency is as high as 67/124/283ms, making it only suitable for scenarios involving a massive amount of data.

Hardware-software (HW/SW) co-design for XMSS and SLH-DSA were explored in [15] and [16]. They integrated specific hash accelerator on RISC-V processor to speed up the most compute-intensive operation. The other processes are executed in software. These works achieved high flexibility with long computation time.

In comparison to SW, HW/SW methods, hardware design often offers less operation cycles, resource overhead and higher efficiency. *Amiet D et.al* presented two high-speed implementations for SPHINCS and SPHINCS+ to meet requirements in the internet field of 5G era [17], [18]. They analyzed the critical path on FPGA platform and inserted significant amounts of registers to increase frequency. Memory footprint is optimized in [19] to meet the stringent resource requirements, at an expense of efficiency reduction. Hardware implementation of BDS (Buchmann, Dahmen, and Schneide) algorithm is instantiated in previous XMSS processor [20], [21]. It accelerates the signature generation in *Sign*, while key generation, which costs the longest time in XMSS, has not been optimized. *Thoma and Güneysu* [22] conducted a comprehensive evaluation for different hash cores and *WOTS+* chain parallelism. They proposed three XMSS processors independently from the perspectives of high performance, low resource overhead and time-area compromise. However, this static reconfigurable design lacks the ability to change security parameters in real time. Another XMSS leaf generator using 28nm bulk industrial CMOS process is presented in [23]. It achieves a high frequency of 1 GHz, but the entire procedure of digital signature cannot be completed.

The above hardware designs are all targeting on a specific domain for SPHINCS+ and its related algorithms. However, different scenarios may put forward diverse security requirements. For example, in wireless applications such as Bluetooth and Wi-Fi, connections between two devices require shorter data transmission for lower latency and general security, while operations related to financial payments focus on higher security. This makes crypto-processor need to be compatible with different security levels.

This work proposes an efficient and reconfigurable SPHINCS+ processor. It achieves high hardware efficiency and flexibility benefit from the optimization methods. The contributions are listed as follow:

- A unified chain address generator and on-the-fly *WOTS+* public key generation scheme are put forward. They accelerate the time-intensive operation through efficient resource utilization in parallelism and execution.
- We propose a security switch mechanism to support diverse parameter sets of SPHINCS+. This scheme addresses the bit misalignment among different data lengths by simple shift operations and saves the logic resources overhead.
- Towards the shortcomings of long intermediate variables and signature, several storage optimization methods including bandwidth analysis, grouped subtree and segmented signature streaming are applied. The memory consumption is reduced to 16k bytes.
- This efficient and reconfigurable SPHINCS+ processor is implemented on Artix-7/Zynq Ultrascale+ FPGA and has achieved high hardware efficiency. It supports SPHINCS+ for three security levels and performs 4%/58% better in ATP for *Sign/Verify* than the other leading designs.

The remainder of this paper is organized as follows. Section II illustrates SPHINCS+ algorithm with related hash-based signature. Section III describes the detailed hardware architecture with optimization strategies. Experimental results and comparison with relative works are given out in Section IV. Finally, section V concludes this paper.

II. Preliminary

Every HSS utilizes a sponge construction hash function to sign and verify message. The hash function absorbs message of arbitrary length and produces n bits digest. Here, we note it as F: $\{0, 1\}^* \rightarrow \{0, 1\}^n$.

SPHINCS+ is a stateless state-of-the-art hash-based signature scheme. It achieves a good balance between performance and resource overhead. Four n bits keys: $\{\boldsymbol{sk.seed}, \boldsymbol{sk.prf}, \boldsymbol{pk.seed}, \boldsymbol{pk.root}\}$ are defined in this algorithm, the first three are all generated by pseudorandom number generators. The $\boldsymbol{sk.seed}/\boldsymbol{pk.seed}$ serve as random seed to generated secret/public keys. Message digest is obtained through a hash function with $\boldsymbol{sk.prf}$. Finally, $\boldsymbol{pk.root}$ is the tree root of top *WOTS+* layer as described later.

Multiple hash algorithms are instantiated in this PQC scheme, mainly including Merkle tree [24], *WOTS+* and forest of random subsets (*FORS*). They are illustrated as follow.

A. Merkle Tree

Lamport [25] proposed the first method for constructing digital signatures with hash functions in 1979. It is also known as Lamport–Diffie one-time signature scheme (*LD-OTS*), which later became the basis of many HSS. In *LD-OTS*, a secret key and its hash digest namely public key are generated for 1 bit message: To sign bit 0, the signature choses secret key. To sign bit 1, choses public key. Although this *OTS* can provide promised security, the secret key can only be used once and it's not conducive to practical application. A tree structure was introduced by *Ralph Merkle* to solve the problem. His main idea lies on reducing the validity of keys to a public one, which is the root of hash tree.

A Merkle tree of height H contains 2^H leaves and signs H bits message. Before the signing procedure begins, 2^H leaves need to be generated, they represent the tree height $= 0$. Then, internal nodes in the next are calculated following its construction rule: parent node is the hash digest of its left and right child nodes, as shown in formula (1). This operation continues until tree height reaches H, where only one node

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on September 25,2026 at 09:10:46 UTC from IEEE Xplore. Restrictions apply.

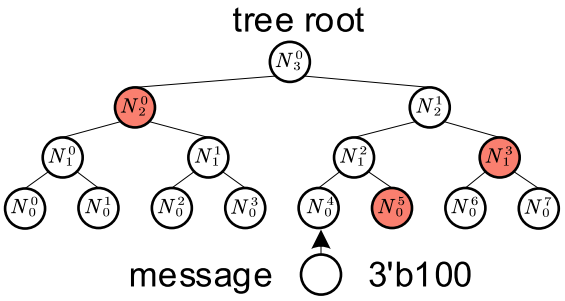


Fig. 1. Merkle tree with height = 3.

Algorithm 1 WOTS+ Sign

Input: $m \in \{0,1\}^*$
Output: $\sigma \in \{0,1\}^{n*t}$, $Y \in \{0,1\}^{n*t}$
Parameter: ω, n, t_1, t_2, t
01: $M = H(m)$
02: $X = \{0,1\}^{n*t}$
03: $M_i = M\left[(i+1)*\omega - 1, i*\omega\right]; i \in [0, t_1 - 1]$
04: $P = \sum_{i=0}^{t_1-1} 2^{\omega} - M_i$
05: $Q = P||M$
06: $Q_i = Q\left[(i+1)*\omega - 1, i*\omega\right]; i \in [0, t-1]$
07: $X_i = X\left[(i+1)*n - 1, i*n\right]; i \in [0, t-1]$
08: $\sigma_i = F^{Q_i}(X_i)$
09: $Y_i = F^{2^{\omega}-1}(X_i)$

remains, that is, the public root of a Merkle tree.

$$N_h\left[j\right] = F\left(N_{h-1}\left[2j\right] \parallel N_{h-1}\left[2j+1\right]\right) \tag{1}$$

For H bits message m, a leaf is selected based on its value. Then the signature is the complementary nodes of this leaf and its higher parent ones, also known as authentication path. During message verification, there is no need to recalculate all the nodes in this Merkle tree. Instead, authentication path is utilized to compute tree root quickly. Finally, the recomputed one is compared with public root to verify their consistency. This tree structure allows for signing multiple messages with limited secret keys and we call it many-time signatures (*MTS*).

A Merkle tree with height = 3 is shown in Fig. 1. When the 3 bits message m equals to 3'b100, leaf N_0^4 is selected, and its authentication path (red nodes) is $\{N_0^5, N_1^3, N_2^0\}$. During the verification phase, only three hash operations are required to compute the root: $N_1^2 = F(N_0^4||N_0^5)$, $N_2^1 = F(N_1^2||N_1^3)$ and $N_3^0 = F(N_2^0||N_2^1)$.

B. WOTS+ and WOTS+ Subtree

WOTS+ is derived from *WOTS*, which balances computational efficiency and signature length. It owns the ability to against collision and pre-image attacks. The central idea is dividing message into several segments and performing hash operations based on the secret key. As the iterations equal to the value of each segment, multiple bits can be singed simultaneously. This improves operational efficiency. The details of *Sign* procedure are outlined in Algorithm 1.

WOTS+ defines parameters ω, n, t_1, t_2 and t. ω represents the number of bits in each segment; n is the length of a massage as well as *WOTS+* secret/public key; t_1, t_2 and t are defined as $t_1 = \lceil n/\omega \rceil, t_2 = \left\lceil \left(\left\lfloor \log_2 t_1 \right\rfloor + 1 + \omega \right)/\omega \right\rceil, t = t_1 + t_2$. For n bits message M, it is first divided into t_1 groups of ω bits M_i, where $i \in [0, t_1 - 1]$. The next step is the **check_sum**, which involves summing up each segment of 2^{ω}-1 - M_i to obtain $t_2 \times \omega$ bits string P. Finally, M and P are concatenated to form $t \times \omega$ bits $Q = (Q_0, Q_1 \cdots Q_{t-1})$, completing the message segmentation.

Before signing these segmented information, *WOTS+* secret keys $X = \{X_{t-1}, X_{t-2} \ldots X_0\}$ are generated from ***sk.seed***. Then, one set of public keys $Y = \{F^{2^{\omega}-1}(X_{t-1}), \ldots F^{2^{\omega}-1}(X_0)\}$ and signatures $\sigma = \{F^{Q_{t-1}}(X_{t-1}), F^{Q_{t-2}}(X_{t-2}) \ldots F^{Q_0}(X_0)\}$ are computed. Here, $F^{i+1}(X) = F(F^i(X))$ and $F^0(X) = X$, is known as the *WOTS+* chain. This process requires ***pk.seed*** as input data.

WOTS+ tree is a simple instantiation of Merkle tree with height = r. It requires to obtain 2^r sets of *WOTS+* public keys. Then each set is compressed with hash function to generate a *WOTS+* leaf.

The verification consists of two steps. At first, each signature σ_i is computed $2^{\omega} - 1 - Q_i$ times to reconstruct Y_i and all public keys are compressed to generate the selected *WOTS+* leaf. In the second step, one can recompute tree root through authentication path efficiently.

C. FORS

FORS is a combination of *LD-OTS* and Merkle trees with large scale. This margin enhances security and allows a single key to sign multiple messages. The advantage comes at a cost of longer signature and processing time.

FORS defines parameters k and a for signing $k \times a$ bits. It consists of $k \times 2^a$ secret keys, which are generated through hash operation with ***sk.seed***. Then, these are divided into k sets and each set are further hashed to generate 2^a *FORS* public keys. They serve as the leaves to instantiate a Merkle tree. Once all the tree roots are computed, they are hashed into n bits digest, namely *FORS* combined root.

When signing a message of arbitrary length, the first step is to obtain $k \times a$ bits hash digest with ***sk.prf***. Then its every a bits correspond to a *FORS* tree and the correlated authentication path serves as the signature.

D. SPHINCS+

SPHINCS+ instantiates a hypertree structure as depicted in Fig. 2. Each hash operation needs 256 bits ***tree addr*** as part of input data. The ***tree addr*** contains 6 components: *layer*, *tree address*, *type*, *keypair*, *tree height* and *tree index*. These addresses describe the position of within the hypertree [6].

An arbitrary message is hashed to generate $k \times a + D \times r$ (msg_1+msg_2) bits digest at first. Then msg_1 goes through *FORS Sign* to generate *FORS* combined root and corresponding signature. After that, it enters layer 0 of *WOTS+*. In this layer, *FORS* combined root and the first r bits of msg_2 are signed in *WOTS+* and its subtree respectively. Then, each subsequent layer signs the root of previous and next r bits of msg_2. The progress continues until reaching the top layer, whose root serves as ***pk.root***.

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on September 25,2026 at 09:10:46 UTC from IEEE Xplore. Restrictions apply.

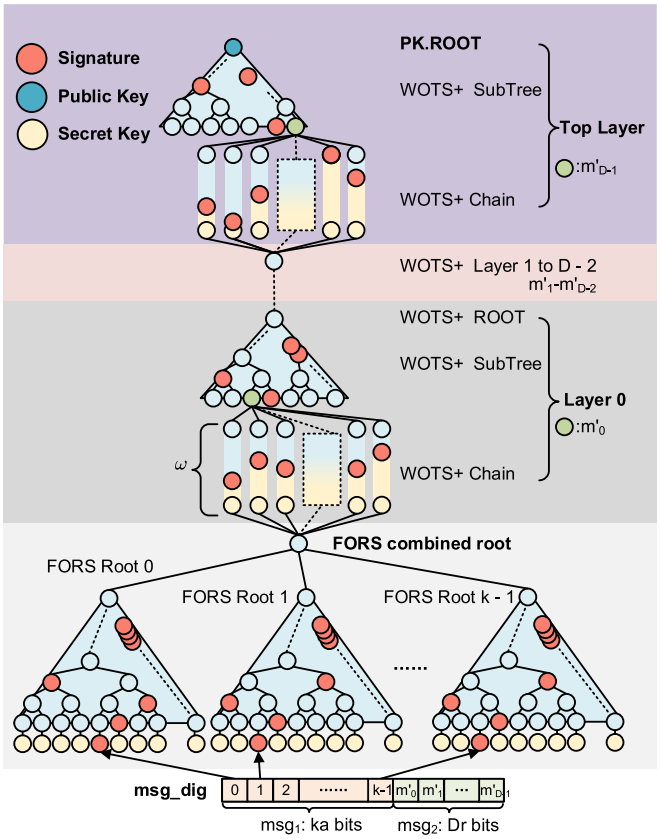


Fig. 2. SPHINCS+ Hypertree.

TABLE I
PARAMETERS SET IN SPHINCS+

	n	t	ω	k	a	r	D
SPHINCS+128	128	35	4	33	6	3	22
SPHINCS+192	192	51	4	33	8	3	22
SPHINCS+256	256	67	4	35	9	4	17

Note that every *WOTS+* layer is independent in the hyper tree, it is possible to compute the top in *Keygen* and obtain ***pk.root*** directly. This improves computational efficiency compared to XMSS. The latter needs to start from the lowest and progress layer by layer.

This paper focuses on the 3 security levels of SPHINCS+. The specific parameter sets $(n, t, \omega, k, a, r, D)$ are shown in Table I.

III. HARDWARE DESIGN

A. Overall Architecture

The overall architecture is shown in Fig. 3, including control unit, hash input logic, KECCAK core, memory, signature generator, and I/O interface. Control unit generates control signals and ***tree addr*** for each module during the process. Hash input logic provides 1088 bits valid data for KECCAK core so as to complete hash operations. Memory adopts 512 bits reg-file to store the most frequently used ***sk/pk.seed***. Intermediate variables are saved in 64 × 1024 bits data RAM through data FIFO to achieve width matching for different security levels. Expected signatures are judged by signature generator and stored into sig RAM through sig FIFO. Then they are transmitted through I/O interface in time.

B. KECCAK Core

SHAKE256 [26] is selected as the basic operation in this design. The algorithm is a member of SHA-3 family and completes state permutation through KECCAK iteration. The stronger obfuscation mechanism enhances its resistance against exhaustive, collision and preimage attacks. Therefore, NIST recommends this algorithm over the other options in official document [27], [28].

A KECCAK core is designed to speed up the SHAKE256 operations. It instantiates an 8-round KECCAK permutation for fewer cycles and inserts four 1600 bits buffers evenly to shorten the critical path. These buffers support the module to perform four hash operations simultaneously. This simplifies the control logic of Merkle tree as each height contains a power of two hash operations. Moreover, since $t = 35/51/67$ public keys need to be generated in each *WOTS+*, the parallelism ensures decent performance when computing the first 32/48/64 and enables us to calculate the last 3 with relatively high hardware efficiency.

When carrying out four SHAKE256 operations at the same time, 12 cycles are required to complete 24 rounds of the KECCAK iteration. Among them, 4 cycles are utilized for taking in new input data and in the meanwhile, the four previous results are output. The idle cycles provide ample time for new data preparation and results storage, as discussed in sections III-D and III-E.

C. Unified Chain Address Generator

The public key generation in *WOTS+* takes up over 80% of the hash executions and is the most significant step that deserves optimization. During this process, ensuring the KECCAK core to perform four hash operations concurrently will achieve high hardware efficiency. Two variables {***chain index***, ***pk index***} are defined to locate a hash operation. The former specifies the location in a chain, while the latter represents the index of public key. They serve as part of the ***tree addr***, namely ***chain addr*** together. In *Sign* procedure, the process contains 2 stages: Stage I generates X_i with ***chain index*** $= 0x0$ and ***sk.seed***; Stage II computes $Y_i = F^{15}(X_i)$ with ***chain index*** starts from $0x0$ to $0xE$ and ***pk.seed***. Then, if the ω-based representation $Q_i = 0x0$, X_i in stage I is the corresponding signature. Otherwise, σ_i is the result when ***chain index*** $= Q_i$-1. This process means ***chain index*** changes with the same rule for every ***pk index*** and the consistency is convenient for hardware implementation. However, during the *Verify* phase, ***chain index*** starts from Q_i to the same end $0xE$ as $Y_i = F^{15-Q_i}(\sigma_i)$ is computed. The different start point and corresponding number of operations pose a challenge in computing multiple public keys simultaneously. A unified chain address generator is introduced to solve this issue as shown in Fig. 4(a).

This module is the core part of control unit and contains 4 banks, each generating one set of ***chain addr***

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on September 25,2026 at 09:10:46 UTC from IEEE Xplore. Restrictions apply.

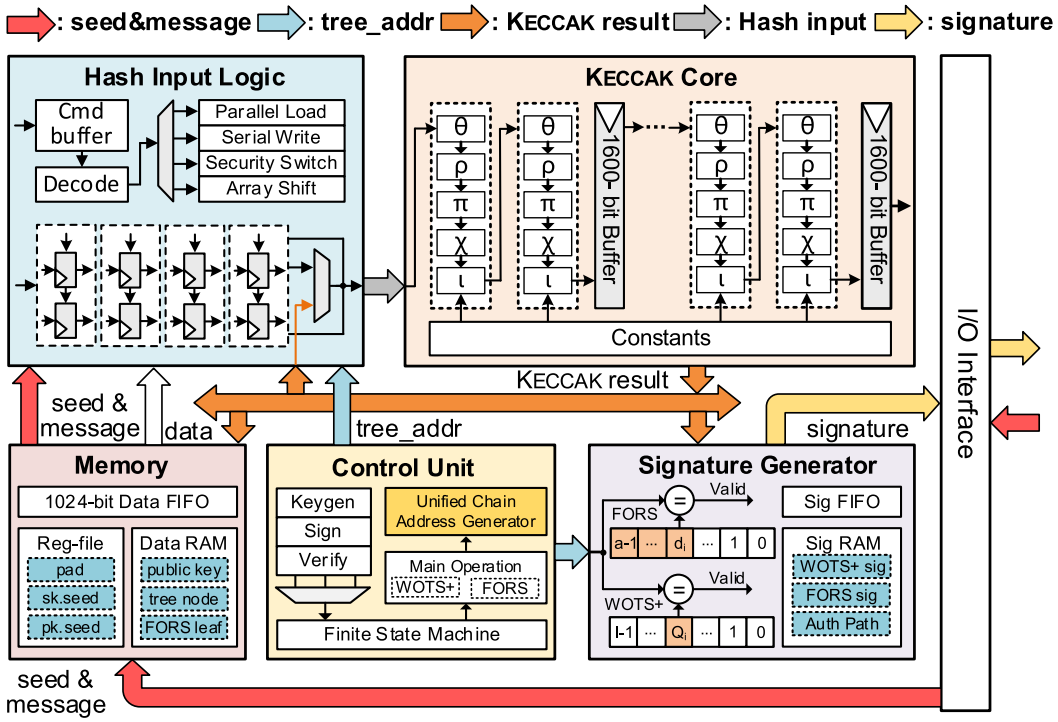


Fig. 3. Overall architecture of SPHINCS+ processor.

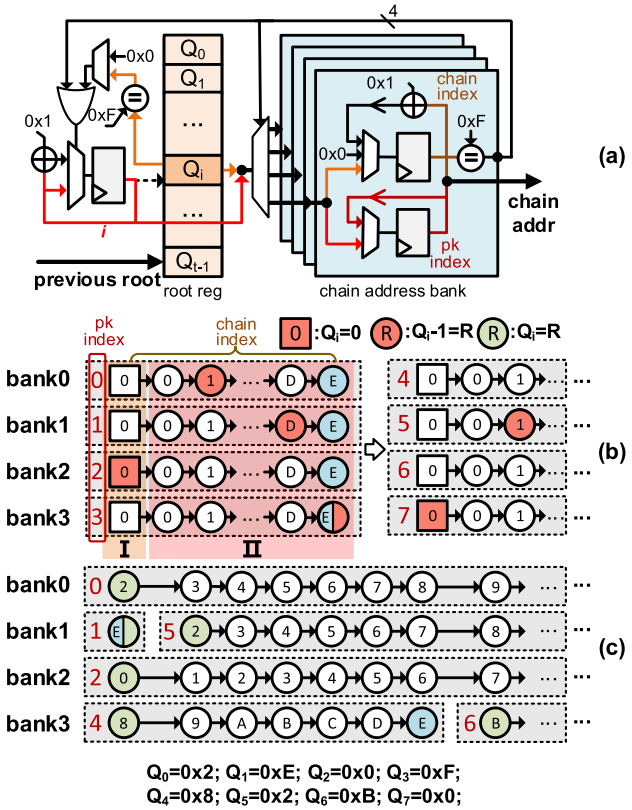


Fig. 4. (a) Unified chain address generator (b) Chain address generation in sequential mode (c) Chain address generation in out-of-order mode.

for KECCAK core. Then a register $\boldsymbol{i}$ is utilized to keep track of the generated public key. Root in previous *WOTS+* layer is stored in ***root reg*** for the *Verify* process. The module generates ***chain addr*** in two modes as shown in Fig. 4(b)(c). We take $Q_0 \sim Q_7 = \{0x2, 0xE, 0x0, 0xF, 0x8, 0x2, 0xB, 0x0\}$ for an example. Each square/circle represents the generated ***chain addr*** in stage I/II for corresponding hash operation. The gray dotted box means one *WOTS+* chain. In sequential mode for *Keygen/Sign*, ***chain index*** changes with previous rule until reaching $0xF$. The value $0xF$ is abandoned and a control signal is produced for $\boldsymbol{i}$ to accumulate. Then, {***chain index***, ***pk index***} is updated as $\{0x0, \boldsymbol{i}\}$. During above process, all the signatures are recorded. In out-of-order mode for *Verify*, the four banks run completely independently. At first, the validity of $Q_{\mathbf{i}}$ needs to be checked. Because when it is $0xF$, current signature equals to the relative public key and no computation is needed. In this case, $\boldsymbol{i}$ just adds one until $Q_{\mathbf{i}}$ is less than $0xF$ as shown in Fig. 4(c). The {***chain index***, ***pk index***} for bank0 to bank2 are loaded as $\{Q_0 = 0x2, 0\}$, $\{Q_1 = 0xE, 1\}$, $\{Q_2 = 0x0, 2\}$ at first. However, since $Q_3 = 0xF$, bank3 is initialed as $\{Q_4 = 0x0, 4\}$ for regenerating Y_4. After one hash operation, bank1 is the first to complete a *WOTS+* chain and updated as $\{Q_5 = 0x2, 5\}$. The process continues until all the Y_i is reconstructed.

With this module, we ensure that 4 ***chain addr*** are ready before the next hash operation and then KECCAK core can perform 4 computations simultaneously. This achieves efficient resource utilization.

D. Hash Input Logic

The hash input logic (HIL) absorbs various components and generate corresponding input data for each hash operation. Fig. 5 shows the architecture of this module. We instantiate 4 register arrays ($array_{0-3}$) to store the 4 KECCAK input data.

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on September 25,2026 at 09:10:46 UTC from IEEE Xplore. Restrictions apply.

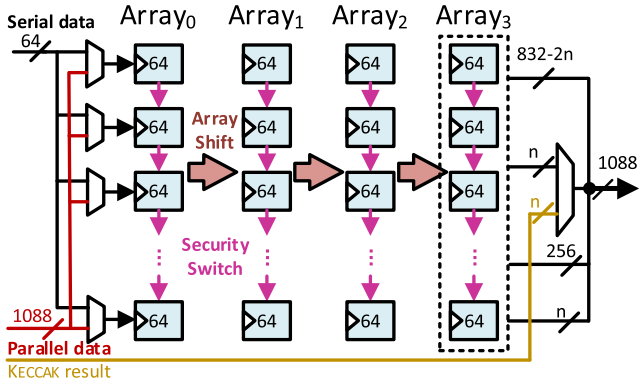


Fig. 5. Hash input logic.

Each array contains 1088 bits, which is determined by the block size of SHAKE256. The 1088 bits can be further subdivided into 17 groups, with each group containing 64 registers due to the maximum common factor for $n = 256/192/128$.

1). Security switch mechanism

This module supports 4 types of operation: **Parallel Load**(**PL**), **Serial Write**(**SW**), **Security Switch**(**SS**) and **Array Shift**(**AS**). The first two is only executed in $array_0$. In **PL** operation, n bits key with 256 minus n (256 - n) bits redundant in reg-file, 256 bits ***tree addr*** provided in control unit, related pad and empty information are parallel written into $array_0$. This step is important for two reasons. Firstly, through introducing redundant data for $n = 192/128$, a relatively fixed hardware connection is ensured, and logical resource overhead is reduced. Secondly, for an efficient utilization of KECCAK core, four input data need to be generated within 12 clock cycles. The parallel input method greatly ensures this requirement. During **SW** operation, intermediate variables are read from memory and then written to $array_0$, this operation is mainly performed in Merkle tree and the generation of *WOTS+* leaf. Then, **AS** is involved by register shifting among $array_0$ to $array_3$. The operation allows $array_0$ to prepare for the next data. When repeating the above operations for four times, **SS** is operated for security switch. Because in **PL** operation, 256 bits key with superfluous data is directly written into $array_0$, 0/64/128 bits redundant are involved in SPHINCS+256/192/128 respectively, they are eliminated by shifting operation among groups in each array.

2). On-the-fly WOTS+ public key generation

We will illustrate the above operation with the generation of *WOTS+* public key, which accounts for the majority of the hash executions. The detailed procedure for $array_3$ is shown in Fig. 6. And numbers represent the data width of each component.

In **PL** step, $array_0$ is directly parallel loaded following the standard of SPHINCS+256, including n bits reserved portion (highlighted in green), n bits key (highlighted in yellow), $512 - 2n$ bits redundant (highlighted in blue), 256 bits ***tree addr*** and relative pad. Subsequently, since this process does not require data retrieval from memory, it proceeds directly to the **Array Shift** phase. After 4 **AS** operations, all the four arrays store the input data with redundant. Then during the **Security Switch** step, for $n = 192/128$, these arrays require 2/4 **SS** operations to eliminate superfluous data caused by key and reserved portion. Finally, since input data for *WOTS+* public key generation relies on previous result, an on-the-fly generation scheme is adopted to take advantage of the computation resource. Reserved portion in $array_3$ is replaced by n bits previous KECCAK result and this eliminates operations for storage/fetch.

Taking the most complex process, SPHINCS+128, as an example. Fig. 7 shows the scheduling of HIL and KECCAK core in time dimension. It requires 8 clock cycles to perform 4 **PL/AS/SS** operations to prepare input data in step 1. Since the two both modules own 4 register arrays, step 2 transforms the data to KECCAK core in 4 **AS** and the next 4 hash input are taken in with **PL** operations simultaneously. Then, HIL complete 4 **SS** operations during the KECCAK iteration. After that, KECCAK core absorbs the new data with previous result and at the same time, HIL starts to prepare data for next round. Note that KECCAK core runs in a pipeline mode throughout the process, it still performs hash operation in step 2/4. This scheme improves hardware efficiency as an uninterrupted operation in time dimension is guaranteed.

E. Memory Optimization

Several strategies for memory optimization are presented in this work. They make a good balance between memory bandwidth and performance while reducing the storage overhead.

1) Bandwidth Analysis: A detailed analysis on the characterizations of main computation processes is conducted for efficient memory scheduling. The results for *Sign* stage of SPHINCS+256 are shown in Fig. 8.

Fig. 8(a) displays the percentage of theoretical hash counts in the main procedures. The public key generation of *WOTS+* and *FORS* share the same format and together they account for approximately 92.9% of the computational cycles, while Merkle trees in *WOTS+* and *FORS* occupy about 4.89% together. Fig. 8(b) represents the percentage of memory access including reading and writing for each hash operation. Since we compute *WOTS+* public key with an on-the-fly strategy, only the last result needs to be stored, totally accounting for only 17.5% of the memory access. On the other hand, each hash execution in *FORS* tree requires reading/writing 512/256 bits data, it takes up the majority of memory access.

Furthermore, we have defined average memory access for SPHINCS+256 as (number of memory access $\times$ 256) / number of hash executions. Its result means the amounts of bits needed to be accessed during one hash operation, as shown in Fig. 8(c). We can find that the value of *WOTS+*/*FORS* public key generation is only 16/128 bits per operation (bits/op), while the others are higher than 768 bits/op. It means that approximately 92.9% of the hash operations are unlikely to be limited by memory bound. And increasing bandwidth will not improve system performance significantly.

Based on the above conclusions, we instantiate a 64×1024 bits data RAM to store intermediate variables through 1024 bits data FIFO. This memory bandwidth can complete data reading/writing in an integer number of accesses, which is hardware friendly. Besides, as one SHAKE256 takes 3 clock cycles on average, the data RAM

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on September 25,2026 at 09:10:46 UTC from IEEE Xplore. Restrictions apply.

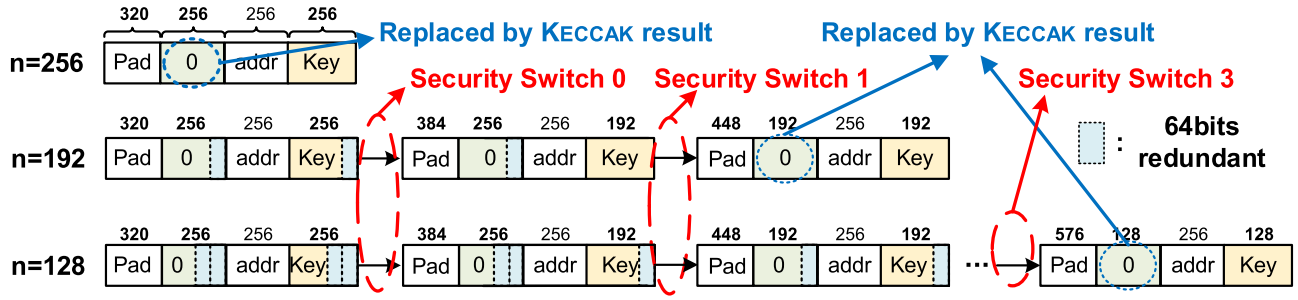


Fig. 6. Security switch process of *WOTS+* in SPHINCS+ 256/192/128.

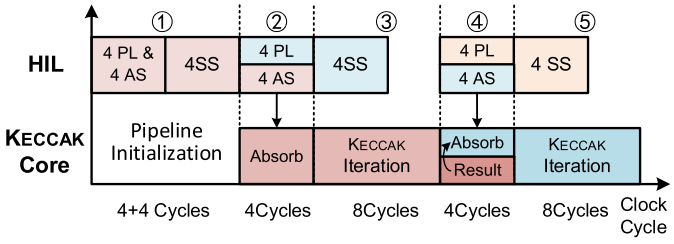


Fig. 7. Module scheduling of *WOTS+* in SPHINCS+128.

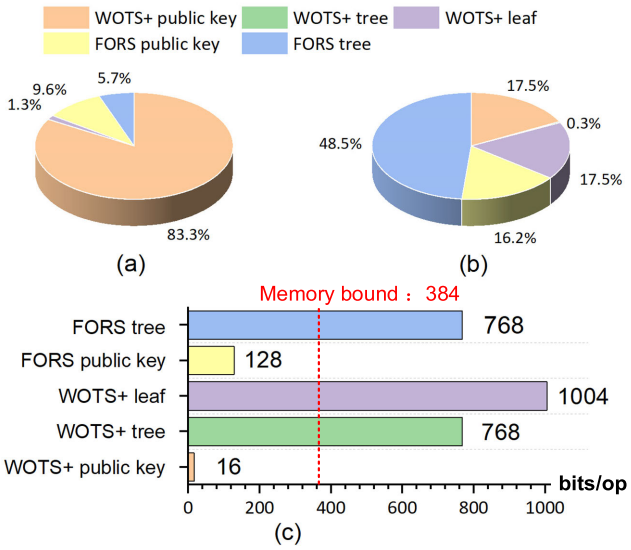


Fig. 8. Characterization of main processes. (a) Number of hash execution. (b) Number of memory access. (c) Average memory access.

can support 384 bits/op of the average memory access. This ensures the most time-consuming operation will not be limited by memory bound. Compared with previous work that utilize DPRAM with 128 bits data width to achieve reading tree node in one cycle [17], [18], our design offers higher efficiency.

2) Grouped Subtree: Fig. 9 shows a grouped subtree scheme towards the *FORS* tree with height = 9 in SPHINCS+256. It contains two stages: in stage I, nodes in the lower 7 levels are divided into 4 subtrees, each containing 2^7 leaves. These leaves take up half of the data RAM. As all leaves are prepared, it's convenient to calculate up level by level until to the subroot, which is saved in the other half of the space. After four subroots are ready, a tree with height = 2 is computed in stage II, completing the large *FORS* tree.

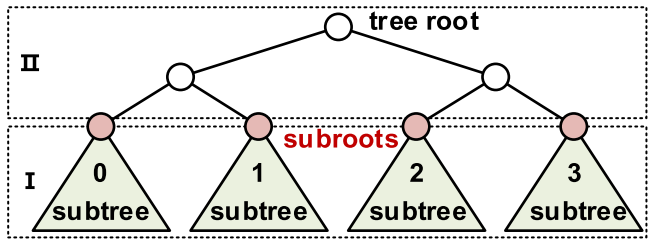


Fig. 9. Grouped subtree for *FORS*.

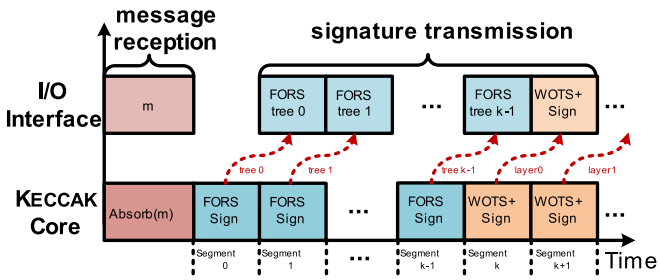


Fig. 10. Segmented signature streaming.

The scheme addresses the issue that data RAM is not enough for storing all the 2^9 leaves. And since this process always computes tree nodes in level $\boldsymbol{h}$ after the completion of level $\boldsymbol{h-1}$, the control logic has been simplified compared with conventional treehash algorithm [6].

3) Segmented Signature Streaming: Another challenge for hardware implementation is the large signature. It results in high storage consumption and transmission latency. We employ the feature that signature is generated evenly over time and introduce a segmented signature streaming strategy.

Fig. 10 shows the data flow of *Sign* phase. At first, message is accepted and absorbed as per 136 bytes. This method prevents processor to store all the information and can generate signature for data of arbitrary length. Then the procedure is divided into several segments according to each *FORS* tree or *WOTS+* layer. In each segment, previous signature is transmitted through I/O interface while the KECCAK core performing hash function concurrently. Thus, we only need to store at most 2×2272 bytes valid signature. A 64×1024 bits DPRAM (sig RAM) is utilized for the above operation, as one port for signature saving while the other for transmitting. Instead of consuming 14 BRAMs to store the entire 49856 bytes valid signature, this scheme has achieved about 90% storage reduction and saves the transmission time in practical application.

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on September 25,2026 at 09:10:46 UTC from IEEE Xplore. Restrictions apply.

TABLE II
LATENCY OF PROCESSOR ON ARTIX-7/ ULTRASCALE + FPGA

n	*Keygen* (kCCs/ms[a]/ms[b])	*Sign* (kCCs/ms[a]/ms[b])	*Verify* (kCCs/ms[a]/ms[b])
128	14.7/0.082/0.045	349.0/1.950/1.074	27.7/0.154/0.085
192	21.4/0.120/0.066	571.9/3.195/1.760	40.5/0.226/0.124
256	56.6/0.316/0.175	1183.7/6.613/3.642	43.1/0.244/0.132

a: Results for Artix-7 FPGA.
b: Results for Ultrascale + FPGA.

TABLE III
EXECUTION CLOCK CYCLES OF MAIN PROCESS IN SPHINCS+256

Execution	*Sign* (CCs/%)		*Verify* (CCs/%)	
WOTS+ tree	3196	0.27%	1224	2.84%
WOTS+ leaf	67184	5.68%	4199	9.73%
WOTS+ public key	891344	75.30%	30005	69.55%
FORS public key	101838	8.60%	-	0%
FORS tree	117250	9.91%	6550	15.18%
Others	2876	0.24%	1162	2.70%
Total	1183688	100.00%	43140	100%

TABLE IV
RESOURCE CONSUMPTION OF MAJOR MODULES ON ARTIX-7 FPGA

Modules	LUT	FF	BRAM
Control unit	**2507/8.5%**	**391/2.8%**	**0**
Signature generator	**1091/3.7%**	**1046/7.4%**	**2**
└ Sig RAM	0	0	2
Hash input logic	**3888/13.2%**	**4558/32.3%**	**0**
KECCAK core	**20966/71.3%**	**6420/45.6%**	**0**
Memory	**958/3.3%**	**2693/11.9%**	**2**
└ Data FIFO	958	1163	0
└ Data RAM	0	0	2
└ Reg-file	0	512	0
Total	29410	14090	4

IV. RESULTS AND COMPARISONS

A. Results

The above SPHINCS+ processor is described using verilog HDL and verified through Modelsim. Then, we implement it on Artix-7 XC7A100T and Zynq Ultrascale+ XCZU3EG FPGA to provide a fair comparison. On Artix-7/Ultrascale+ device, the design consumes 29410/29172 LUTs, 14090/14076 FFs, 4/4 BRAMs with no DSP. The maximum frequencies are 179/325 MHz, respectively. The other details are shown in Table II, III and IV.

Table II shows the latency for different security levels. When n quals to 128, 192, 256 the operation time for *Keygen*/*Sign*/*Verify* is 0.082/1.950/0.154ms, 0.120/3.195/ 0.226ms, 0.316/6.613/0.244ms on Artix-7 FPGA and 0.045/1.074/ 0.085ms, 0.066/1.760/0.124ms, 0.175/3.642/0.132ms on Ultrascale+ FPGA. Due to a fixed number of hash operations, the latency for *Keygen*/*Sign* is constant and will not change with the variation of public/private keys. As for *Verify*, above results are the average after 10000 runs. This is because signature determines the length of each *WOTS*+ chain, which makes process time vary within a small range. Although our design is not a constant-time implementation for this reason, all the data in *Verify* is public and the procedure will not leak side channel information through operation time.

Actual clock cycles for SPHINCS+256 are shown in Table III. It can be found that the generation of *WOTS*+ public key in *Sign*/*Verify* costs the most time, accounting for 75.30% and 69.55%, respectively. This is consistent with the theoretical analysis in Section III. Benefit from the proposed optimizations, four hash operations are executed with no bubbles. And the time-intensive operation is greatly accelerated.

Resource consumption on Artix-7 FPGA is shown in Table IV, the results are nearly the same on Ultrascale+ device. It is worth mentioning that hash input logic consumes 13.2% of the LUT and is able to generate the input data for all security levels of SPHINCS+. This is achieved through our security switching mechanism, which utilizes simple shift operations with ample cycles to reduce logic resources. Furthermore, we can find that KECCAK core occupies majority of the system resources, consuming 71.3%/45.6% in LUT/FF, while the control unit accounts for only 8.5%/2.8%, this demonstrates the simplicity of SPHINCS+. In scenarios where high security requirements are not necessary, selecting lightweight hash functions can reduce the resource overhead of the system.

B. Comparison With Related Works

We conduct a comparison with the existing works, focusing on the implementations of SPHINCS+ (or SLH-DSA), SPHINCS and XMSS. The details are shown in Table V, including resources consumption, max frequency and execution time for *Keygen*/*Sign*/*Verify*. Unless otherwise specified, the following conclusions are derived from a comparison of $n = 256$.

Optimized software-based benchmark is implemented in 3.1 GHz Intel Xeon E3-1220 CPU with AVX2 instructions [6]. Compared to this work, our implementation is 8.7×/8.6×/6.6× faster in *Keygen*/*Sign*/*Verify*, demonstrating the excellence of hardware implementation. *Sun et al.* [14] utilize the NVIDIA GeForce GTX 1080 GPU for accelerating KECCAK computations. They execute *Keygen*/*Sign*/*Verify* operations for 4096/256/16384 parallelization, and achieve the highest speed of 61134/2065/57894 ops among existing designs. However, its single-operation latency is 212×/19×/1159× slower than our implementation. Therefore, this design is primarily suitable for scenarios with extremely high data throughput, where simultaneous computations of multiple *Sign*/*Verify* are required.

Previous high-speed SPHINCS+ accelerators for different security levels are implemented independently in [18]. They adopted 12-stage KECCAK permutation and massive register insertion to cut off critical path. Thus, their KECCAK core can operate at a max frequency of 500MHz. Moreover, a multi-clock domain scheme is designed as the peripheral control circuit operating at 250MHz. This ensures that a single SHAKE256 operation can be completed in one clock cycle of 250MHz. The approach consumes a significant amount of resource and requires high memory

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on September 25,2026 at 09:10:46 UTC from IEEE Xplore. Restrictions apply.

TABLE V
COMPARISON OF PERFORMANCE AND RESOURCE

	Algorithm	Method	Area					Frequency	Time(ms)/ ATP[b]		
			LUT	FF	BRAM	DSP	Slice[a]	MHz	*Keygen*	*Sign*	*Verify*
This Work (Artix-7)	SPHINCS+ 128	HW	29410	14090	4	0	10043	179.8	0.082/0.08	1.950/1.96	0.154/0.15
	SPHINCS+ 192								0.120/0.12	3.195/3.21	0.226/0.23
	SPHINCS+ 256								0.316/0.32	6.613/6.64	0.244/0.24
[18] (Artix-7)	SPHINCS+ 128	HW	47991	72505	11.5	1	23835	250&500[f]	1.01[c]/2.41[c]	1.01/2.41	0.16/0.38
	SPHINCS+ 192	HW	48398	73476	17	1	25337	250&500[f]	1.17[c]/2.96[c]	1.17/2.96	0.19/0.48
	SPHINCS+ 256	HW	51009	74539	22.5	1	27401	250&500[f]	2.52[c]/6.90[c]	2.52/6.90	0.21/0.58
[16] (Artix-7)	SLH-DSA128	HW/SW	8605	-	32	0	9588	100	1.76/1.68	49/46.0	4.4/4.22
	SLH-DSA192								2.84/2.72	105/101.6	7.11/6.81
	SLH-DSA256								8.15/7.81	236/226.9	8.57/8.21
This Work (Ultrascale+)	SPHINCS+ 128	HW	29172	14076	4	0	9517	325	0.045/0.04	1.074/1.02	0.085/0.08
	SPHINCS+ 192								0.066/0.06	1.760/1.67	0.125/0.12
	SPHINCS+ 256								0.175/0.17	3.642/3.47	0.132/0.13
[19] (Ultrascale+)	SPHINCS+ 128	HW	5917	4933	0.5	0	2153	156	2.02/0.44	64.34/13.86	2.51/0.54
	SPHINCS+ 256	HW	7925	6339	0.5	0	2831	152	8.09/2.29	199.2/56.41	4.55/1.29
[17][d] (Kintex-7)	SPHINCS256	HW	19067	38132	36	0	17899	525	-	1.53/2.74	0.065/0.12
[20] (Artix-7)	XMSS	HW	22300	31215	0	0	9476	110	2880/15.05[e]	15.88/2729[e]	1.45/1.37
[21] (Artix-7)	XMSS	HW	22833	20789	15	0	11792	100	1100/3.54[e]	3/1297[e]	0.68/0.80
[14] (GPU1080)	SPHINCS256	SW	-	-	-	-	-	-	67	124	283
[6] (Haswell)	SPHINCS+256	SW	-	-	-	-	-	-	2.75	57.08	1.62

a: Equivalent Slice is calculated as $LUT/4 + FF/8 + BRAM \times 232.4 + DSP \times 102.4$ for 7 series and $LUT/4 + FF/8 + BRAM \times 116.2 + DSP \times 51.2$ for Ultrascale+ [29][30].
b: ATP is calculated as equivalent Slices × time.
c: Key generation is completed in *Sign* procedure in this design.
d: Using lightweight hash algorithms BLAKE and chacha12.
e: The XMSS parameter {n, h, w} is {256, 10, 16}. And ATPs for *Keygen*/*Sign* are calculated with the time of *Sign*/*Keygen*. Because in XMSS, main tree is calculated in *Keygen*, which corresponds to the *Sign* procedure of SPHINCS+.
f: Multi-clock domains are employed as KECCAK core operating at 500MHz and others at 250MHz. Process time correspond to the both frequencies.

bandwidth for adaptation. Compared to this implementation, our design offers the advantage of flexibility to support three security parameters and reduces resource consumption for 42.3%/81.1%/82%/ 100% in LUT/FF/BRAM/DSP. Furthermore, we have achieved 1.04×/2.41× of the ATP optimizations in *Sign*/*Verify*. This is mainly due to our strategies for the most computationally intensive operation and memory footprint. The reasons why their ATP value for SPHINCS+ 192 *Sign* is better than ours come from two aspects. Firstly, our design supports all the security levels at a main cost of hash input logic. When ignoring the impact of this module, the corresponding ATP is 2.70, which is slightly better. Secondly, their latency is relatively low and close to that of $n = 128$. It can be inferred that the design is more suitable for SPHINCS+192 than the other two security levels. Finally, their accelerator uses *Sign* instead of *Keygen* for key generation. The former incurs nearly 20× of the computation time. This drawback reduces hardware efficiency in practical applications.

Another SPHINCS+ hardware processor specifically targets on embedded devices and IoT applications [19]. The design consumes minimal LUT/FF and its signature streaming mechanism effectively reduced BRAM to 0.5, at the cost of high computational time. Although the design only consumes 70% less equivalent Silce than ours, it runs 46.2×/54.7×/34.5× slower in *Keygen*/*Sign*/*Verify*. As a result, our implementation performs 93%/94%/90% better in ATP.

M.-J. O. Saarinen proposed a HW/SW codesign for SLH-DSA with RISC-V processor [16]. Hash operations are executed in hardware and the programmability allows it to support SLH-DSA with all parameters. This flexibility comes at large storage consumption for firmware and stack. As a comparison, we have achieved about 24.4×/34.2×/34.2× ATP optimization in *Keygen*/*Sign*/*Verify*.

Existing SPHINCS256 processor implemented on 28nm Kintex-7 FPGA selects lightweight hash algorithms BLAKE and chacha12 [17]. These hash algorithms consume less resources and run faster, leading to the high hardware efficiency of this design. SHAKE256 is officially certified by NIST and offers higher resistance to various attacks. When excluding the resource of hash function. It consumes 1.07×/3.08×/9.00× LUT/FF/BRAM compared with our design and lacks the flexibility.

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on September 25,2026 at 09:10:46 UTC from IEEE Xplore. Restrictions apply.

Cao et al. [20] independently implemented XMSS on Artix-7 FPGA with different parameters and result for the lightweight parameters are listed in Table V. Although they proposed an optimized BDS algorithm to accelerate the authentication path generation in *Sign*, their entire process time (*Keygen* + *Sign* + *Verify*) is 405× slower than ours. This is because that a very huge Merkle tree is instantiated in XMSS, while SPHINCS+ adopts a hypertree structure. The latter reduces the defect that operation time increases exponentially with tree height. Another XMSS processor also suffers from long processing times [21], especially for key generation. Our design offers 11×/195×/3× for ATP optimizations with good flexibility.

V. Conclusion

This paper presents an efficient and reconfigurable hardware implementation for SPHINCS+. Through the optimization strategies for the most time-intensive operation, this processor has achieved good performance with limited hardware resources. A comprehensive analysis towards the characterization of algorithm is conducted. Based on the result, methods for efficient memory scheduling and reducing storage consumption are given out. Finally, we propose a security switch mechanism to eliminate the data misalignment among diverse parameters sets. The scheme makes use of the idle cycles to perform related operation and saves logic resource.

To the best of our knowledge, it is the first pure hardware design that supports three security levels ($n = 128/192/256$) of the SPHINCS+ algorithm, making it adaptable to various scenarios with different security requirements.

In future work, we will focus on the potential threat of side-channel attacks and fault attacks in practical application, exploring the efficient integration of defense mechanisms.

References

[1] P. W. Shor, "Algorithms for quantum computation: Discrete logarithms and factoring," in *Proc. 35th Annu. Symp. Found. Comput. Sci.*, 1994, pp. 124–134.

[2] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," *SIAM Rev.*, vol. 41, no. 2, pp. 303–332, Jan. 1999.

[3] L. K. Grover, "A fast quantum mechanical algorithm for database search," in *Proc. 28th Annu. ACM Symp. Theory Comput.-STOC*, 1996, pp. 212–219.

[4] R. L. Rivest, A. Shamir, and L. Adleman, "A method for obtaining digital signatures and public-key cryptosystems," *Commun. ACM*, vol. 21, no. 2, pp. 120–126, Feb. 1978.

[5] F. Pub, "Digital signature standard (DSS)," *Fips Pub*, vol. 2000, pp. 186–192, Jul. 2000.

[6] J.-P. Aumasson et al. (2019). *SPHINCS+−Submission to the NIST Post-Quantum Project*. Accessed: Apr. 2021. [Online]. Available: https://csrc.nist.gov/CSRC/media/Projects/Post-Quantum-Cryptography/documents/round-2/submissions/SPHINCS-Round2.zip

[7] J. Buchmann, E. Dahmen, and M. Szydlo, "Hash-based digital signature schemes," in *Post-Quantum Cryptography*. Berlin, Germany: Springer, 2009, pp. 35–93.

[8] G. Bertoni, J. Daemen, M. Peeters, and G. Van Assche, "Cryptographic sponge functions," Cryptographic Sponge Functions, Jan. 2011. [Online]. Available: https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=6262d2cef0e89b17d8677daf9ddf216a72e88bfa

[9] B. Guido, D. Joan, P. Michaël, and V. Gilles, "The K reference," Jan. 2011. [Online]. Available: https://perso.uclouvain.be/fstandae/source_codes/hash_atmel/specs/keccak.pdf

[10] D. J. Bernstein et al., "SPHINCS: Practical stateless hash-based signatures," in *Proc. Annu. Int. Conf. Theory Appl. Cryptograph. Techn.*, Jan. 2015, pp. 368–397.

[11] A. Hülsing, "W-OTS+−shorter signatures for hash-based signature schemes," in *Proc. 6th Int. Conf. Cryptol. Afr. Prog. Cryptology-AFRICACRYPT*, Cairo, Egypt, 2013, pp. 173–188.

[12] J. Rijneveld, "Implementing SPHINCS with restricted memory," Master's thesis, Inst. Comput. Inf. Sci., Radboud Univ. Nijmegen, Nijmegen, The Netherlands, 2015.

[13] A. Hülsing, J. Rijneveld, and P. Schwabe, "Armed SPHINCS: Computing a 41 KB signature in 16 KB of RAM," in *Proc. 19th IACR Int. Conf. Pract. Theory Public-Key Cryptography*, Taipei, Taiwan, Mar. 2016, pp. 446–470.

[14] S. Sun, R. Zhang, and H. Ma, "Efficient parallelism of post-quantum signature scheme SPHINCS," *IEEE Trans. Parallel Distrib. Syst.*, vol. 31, no. 11, pp. 2542–2555, Nov. 2020.

[15] W. Wang et al., "XMSS and embedded systems: XMSS hardware accelerators for RISC-V," in *Proc. Int. Conf. Sel. Areas Cryptography*, 2019, pp. 523–550.

[16] M. O. Saarinen, "Accelerating SLH-DSA by two orders of magnitude with a single hash unit," in *Proc. Annu. Int. Cryptol. Conf.*, Jan. 2024, pp. 276–304.

[17] D. Amiet, A. Curiger, and P. Zbinden, "FPGA-based accelerator for post-quantum signature scheme SPHINCS-256," in *Proc. IACR Trans. Cryptograph. Hardw. Embedded Syst.*, Feb. 2018, pp. 18–39.

[18] D. Amiet, L. Leuenberger, A. Curiger, and P. Zbinden, "FPGA-based SPHINCS+ implementations: Mind the glitch," in *Proc. 23rd Euromicro Conf. Digit. Syst. Design (DSD)*, Aug. 2020, pp. 229–237.

[19] Q. Berthet, A. Upegui, L. Gantel, A. Duc, and G. Traverso, "An area-efficient SPHINCS+ post-quantum signature coprocessor," in *Proc. IEEE Int. Parallel Distrib. Process. Symp. Workshops (IPDPSW)*, Jun. 2021, pp. 180–187.

[20] Y. Cao et al., "An efficient full hardware implementation of extended Merkle signature scheme," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 69, no. 2, pp. 682–693, Feb. 2022.

[21] J. P. Thoma, D. Hartlief, and T. Güneysu, "Agile acceleration of stateful hash-based signatures in hardware," *ACM Trans. Embedded Comput. Syst.*, vol. 23, no. 2, pp. 1–29, Mar. 2024.

[22] J. P. Thoma and T. Güneysu, "A configurable hardware implementation of XMSS," *Cryptol. ePrint Arch.*, vol. 2021, pp. 352–365, Mar. 2021.

[23] P. Mohan, W. Wang, B. Jungk, R. Niederhagen, J. Szefer, and K. Mai, "ASIC accelerator in 28 nm for the post-quantum digital signature scheme XMSS," in *Proc. IEEE 38th Int. Conf. Comput. Design (ICCD)*, Oct. 2020, pp. 656–662.

[24] R. C. Merkle, "A certified digital signature," in *Proc. Conf. Theory Appl. Cryptol.*, Nov. 2007, pp. 218–238.

[25] L. Lamport, "Constructing digital signatures from a one way function," Dept. Comput. Sci., Lab., SRI Int., Menlo Park, CA, USA, Tech. Rep. CSL-98, 1979..

[26] M. J. Dworkin, "SHA-3 standard: Permutation-based hash and extendable-output functions," Nat. Inst. Standards Technol. (NIST), Gaithersburg, MD, USA, Tech. Rep. FIPS PUB 202, 2015.

[27] S. Kölbl, M. M. Lauridsen, F. Mendel, and C. Rechberger, "Haraka v2-efficient short-input hashing for post-quantum applications," in *Proc. IACR Trans. Symmetric Cryptol.*, 2016, pp. 1–29.

[28] G. Nist, "Secure hash standard (SHS)," NIST, Gaithersburg, MD, USA, Tech. Rep. FIPS PUB 180-4, 2012.

[29] S. Kurniawan, P. Duong-Ngoc, and H. Lee, "Configurable memory-based NTT architecture for homomorphic encryption," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 70, no. 10, pp. 3942–3946, Oct. 2023.

[30] P. Duong-Ngoc, S. Kwon, D. Yoo, and H. Lee, "Area-efficient number theoretic transform architecture for homomorphic encryption," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 3, pp. 1270–1283, Mar. 2023.



Tianze Huang received the B.S. degree in integrated circuit design and integrated system from the Huazhong University of Science and Technology (HUST), Wuhan, China, in 2022, where he is currently pursuing the Ph.D. degree with the School of Integrated Circuits. His current research interests include efficient post-quantum crypto-processor and protection against side-channel attack.

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on September 25,2026 at 09:10:46 UTC from IEEE Xplore. Restrictions apply.



Jiahao Lu received the B.S. and Ph.D. degrees from the Huazhong University of Science and Technology, Wuhan, China, in 2018 and 2023, respectively. He is currently an Post-Doctoral Researcher with the School of Integrated Circuits, Huazhong University of Science and Technology. His current research interests include digital integrated circuits and post-quantum cryptographic processor.



Dongsheng Liu (Senior Member, IEEE) received the Ph.D. degree from the Huazhong University of Science and Technology, Wuhan, China, in June 2007. In 2013, he was selected as Wuhan Chenguang Youth Talent Support Program. He is currently a Full Professor with the School of Integrated Circuits, Huazhong University of Science and Technology. He has been the Team Leader of at least ten important projects in last five years, including a sub-project of the National Science and Technology Major Project, the National Natural Science Foundation of China, the National Key Research and Development Program of China, and the Key Research and Development Project of Hubei Province. He has authored or co-authored more than 70 academic papers and applied for 41 authorized Chinese patents and one authorized American patent. Many of them had been published in the flagship transactions of several IEEE societies, including IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS—I: REGULAR PAPERS, IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS—II: EXPRESS BRIEFS, IEEE INTERNET OF THINGS JOURNAL, IEEE TRANSACTIONS ON INDUSTRIAL ELECTRONICS, IEEE TRANSACTIONS ON INDUSTRIAL INFORMATICS, CICC, ESSERC, ASSCC, and other international top/high-level journal and conferences. Further, his long-tern dedication on the field of IoT is well recognized by the industry community. His research interests include VLSI design, AI processor, and post-quantum cryptographic processor.



Zhixiang Luo received the Bachelor of Engineering degree in automatic control science and engineering and the Master of Engineering degree in electronic and information science and engineering from the Huazhong University of Science and Technology (HUST) in July 1994 and July 1997, respectively, and the Doctor of Engineering degree from the State Key Laboratory of Laser Technology, HUST, in April 2001. Since April 2001, he has been teaching at HUST. In recent years, he has participated in several national and provincial-ministerial projects, such as the National Major Instrument Project "High-Precision Photoacoustic Spectroscopy Detector" and the "Development of Broadband High-Speed Photoelectric Signal Analyzer Equipment." He has published numerous articles in international journals and holds multiple authorized invention patents.



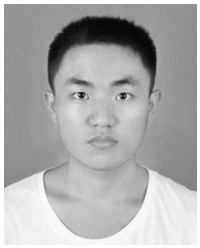
Chi Cheng (Member, IEEE) received the Ph.D. degree in information and communication engineering from the Huazhong University of Science and Technology, Wuhan, China, in December 2013. From 2014 to 2016, he was an International Research Fellow with Japan Society for the Promotion of Science (JSPS), Institute of Mathematics for Industry, Kyushu University, Fukuoka, Japan. He is currently an Associate Professor with the School of Computer Science, China University of Geosciences, Wuhan. His research interests include postquantum cryptography, especially lattice-based cryptography.



Aobo Li (Graduate Student Member, IEEE) received the B.S. degree in optoelectronic information science and engineering from the Huazhong University of Science and Technology, Wuhan, China, in 2020. He is currently pursuing the Ph.D. degree with the School of Integrated Circuits. His current research interests include digital integrated circuit design, hardware security, and post-quantum cryptographic algorithm.



Lei Chen received the B.S. degree from the Huazhong University of Science and Technology (HUST), Wuhan, China, in 2023, where he is currently pursuing the M.S. degree. His current research interests include side-channel attacks and cryptographic processor design.



Shuo Yang received the B.S. degree in integrated circuit design and integrated system from the Huazhong University of Science and Technology, Wuhan, China, in 2021, where he is currently pursuing the M.S. degree with the School of Optical and Electronic Information. His current research interests include digital integrated circuit and post-quantum-cryptography.



Jiaming Zhang is currently pursuing the M.S. degree with the School of Integrated Circuits, Huazhong University of Science and Technology. His current research interests include RISC-V domain-specific architecture and post-quantum cryptography processor design.



Xiang Li received the B.S. degree in integrated circuit design and integrated system from the Huazhong University of Science and Technology, Wuhan, China, in 2021, where he is currently pursuing the M.S. degree with the School of Optical and Electronic Information. His current research interests include embedded systems and coprocessor.

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on September 25,2026 at 09:10:46 UTC from IEEE Xplore. Restrictions apply.